Senior AI Engineer, Quality Platform — Take-Home Assignment

Quality Engineering & AI

Thanks for taking the time to work through this. It's designed to take about 90 minutes to 2 hours of focused work, split across two parts. There's no single "correct" answer we're grading you against — we're interested in how you build AI systems and how you think about quality while doing it.

Time expected: ~90 minutes to 2 hours of focused work (Part A ~60 min, Part B ~30 min). Please don't spend much more than that, we'd rather see clear decisions than exhaustive coverage.

Submission window: Please return this within 5 days of receiving it — take it whenever suits your schedule.

Language: Python preferred; use whatever you're most comfortable with.

Tools: You're welcome to use AI tools (ChatGPT, Claude, Copilot, or anything else). Using them well is genuinely part of this role. Just include the short note described at the end.

LLM access: Part A asks you to call a real model. Use any provider and your own key. If you don't have API access, tell your recruiting contact before you start.

Questions: If anything here is ambiguous, say so in your submission and tell us what assumption you made. Asking a good clarifying question is itself a positive signal.

How we'll evaluate: Mainly how you build and evaluate LLM systems, then how you reason about quality and risk, then how clearly you communicate decisions and trade-offs.

Debrief: After you submit, we'll schedule a 45-minute session where you walk us through your submission and extend your Part A code live.

Part A — LLM Output Evaluator (~60–70 minutes)

Your solution must call a real LLM and use structured output (for example, JSON validated against a schema). Include your entire chat transcripts and the raw model outputs alongside your code.

Include a short write-up (200 words max) on your approach — what you decided, what you left out, and where it would break. The writing matters as much as the code.

The problem

Your team runs a production pipeline that reads a product spec and uses an LLM to generate test cases. The obvious problem: how do you know if the generated test cases are any good?

Here's the spec the pipeline was given:

> *When a business tops up their multi-currency wallet, the system locks the FX rate at the moment the top-up is initiated. The funding source is only debited after the destination wallet is credited. If the payment provider's callback times out after 5 seconds, the transaction is rolled back and neither side is affected. Top-ups above SGD 10,000 require approval from a second business owner before settling.*

And here are four test cases it generated:

```
const generatedTestCases = [
  {
    id: "tc-1",
    description: "Verify that a wallet top-up completes successfully",
    steps: ["Initiate top-up of SGD 500", "Check wallet balance increases"],
    expectedResult: "Wallet balance increases by SGD 500"
  },
  {
    id: "tc-2",
    description: "Verify FX rate is applied correctly",
    steps: [
      "Initiate top-up of USD 100",
      "Check that FX rate shown equals rate at initiation time"
    ],
    expectedResult: "FX rate matches rate locked at initiation"
  },
  {
    id: "tc-3",
    description: "Verify rollback on timeout",
    steps: ["Initiate top-up", "Simulate provider timeout", "Check wallet balance"],
    expectedResult: "Wallet balance unchanged and funding source not debited"
  },
  {
    id: "tc-4",
    description: "Verify large top-up requires approval",
    steps: ["Initiate top-up of SGD 15,000", "Check approval screen appears"],
    expectedResult: "Approval screen is shown"
  }
];
```

- Write an evaluator that scores each test case against the spec — how well it covers the spec, whether it tests the right thing, and whether it would catch a real bug — and explains each score. Use an LLM for at least part of the scoring; combining it with deterministic checks is welcome.
- Run your evaluator 5 times on the same inputs. How much do the scores vary, and what did you

do (or would you do) to reduce that?
- What does your evaluator miss? Write one bad test case that your evaluator scores too highly, and explain why it fools it.
- If this were wired into CI to block merges when generated coverage is poor, what threshold would you set based on the variance that you observed? Give a number, and explain how you'd calibrate it.

Part B — Progressive Scenario (~30–40 minutes)

One scenario, three questions. Each builds on the last — commit to your decisions rather than hedging, because your answer to question 1 shapes what's possible in question 3. Keep each answer to **200 words or fewer**.

Context

Aspire has a merge guardrail that reads product specs from Confluence and blocks PRs when it detects test coverage gaps. You've just been handed ownership of it.

It currently works by keyword matching: it extracts entities from the spec (e.g. "FX rate", "rollback", "approval") and checks whether any test file name or test title (the *describe* / *it* strings) mentions each one. Below are its outputs on three recent PRs.

```
// PR 1
{
  pr: "Add FX rate display caching",
  specEntities: ["FX rate", "cache", "display"],
  testsFound: {
    "fxRate.test.ts": ["locks FX rate at initiation"],
    "display.test.ts": ["renders FX rate to 4 decimal places"]
  },
  verdict: "BLOCKED",
  reason: "No test mentions 'cache'"
}

// PR 2
{
  pr: "Add rollback on callback timeout",
  specEntities: ["rollback", "callback", "timeout", "funding source"],
  testsFound: {
    "rollback.test.ts": ["rolls back wallet credit",
                         "funding source not debited after rollback"],
    "callback.test.ts": ["handles provider callback"],
    "timeout.test.ts": ["timeout is configured to 5000ms"]
  },
  verdict: "PASSED",
  reason: "All spec entities found in tests"
}

// PR 3
{
  pr: "Fix approval threshold for SGD top-ups",
  specEntities: ["approval", "threshold", "SGD", "top-up"],
  testsFound: {
    "approval.test.ts": ["shows approval modal for large top-ups"],
    "topup.test.ts": ["top-up succeeds"]
  },
```

```
  verdict: "BLOCKED",
  reason: "No test mentions 'threshold' or 'SGD'"
}
```

Question 1

For each PR, say whether the guardrail's verdict was right or wrong, and explain specifically how it reached that verdict. Don't describe the general problems with keyword matching — tell us what a correct check would have needed to look at in each case.

Question 2

You've replaced keyword matching with an LLM-based check, and it's ready to deploy. A senior engineer pushes back:

> *"LLMs are non-deterministic. The same PR might pass on Monday and fail on Thursday. We can't block merges on something random."*

How do you respond, what do you build to address the concern, and what would you measure to show it's addressed?

Question 3

Six months in, the guardrail is working well. Engineers now want the next thing: instead of only blocking for missing coverage, can it generate the missing tests automatically? You have the spec, the existing test suite and the code diff. Walk through how you'd build that:

- What context you'd retrieve
- How you'd design the prompt
- How you'd evaluate whether the generated tests are actually useful
- Where you'd expect it to fail

Before You Submit

Please include a short note (2–3 sentences is fine) on which AI tools you used, for which parts, and one place where the AI got something wrong and how you caught it.

Please also share your AI chat transcripts or agent logs (e.g. ChatGPT/Claude share links, or Cursor / Claude Code session exports). We're not looking to penalise AI use — seeing how you prompt, question and correct the AI is part of what we're assessing.

How to Submit

- Part A: a link to a repo/gist, or a zip of your code, including your prompts, raw outputs from all 5 runs, plus the model and settings used.
- Part B: a doc, or plain text in your email.
- Your AI-usage note: Include one place where the AI got something wrong and how you caught it. Include your entire chat transcript

Send everything to your recruiting contact within 5 days. We'll follow up to schedule the debrief shortly after. Good luck, and thanks again for the time.